Week 10
1. UNIX Pipes (I)

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Monday, November 3rd, 2025

Content

Part I
Signals and Files – Review

Signals and files

Some basic Interprocess communication (IPC) can be achieved using:

- Signals: using the `kill()` system call.
- Files: by passing open files across the `fork()/exec()`

The following is an example that illustrates the use of signals and files allowing a child process to provide its parent with data ...

Example 1... (main) (review.c)

```
#include <stdio.h>
#include <signal.h>
#include <unistd.h>
#include <stdlib.h>

void action(int dummy){sleep(1);}
void child(FILE *);
void parent(FILE *, pid_t);

int main(int argc, char *argv[]){
  FILE *fp;
  pid_t pid;
  int childRes;

  fp = fopen("/tmp/ipoc.txt", "w+");
  setbuf(fp, NULL);

  if((pid=fork()) == 0)
    child(fp);
  parent(fp, pid);
}
```

Example 1... (parent)

```
void parent(FILE *fp, pid_t pid){
 int childRes, n=0;

  while(1){
    signal(SIGUSR1, action);
    pause();
    rewind(fp);
    fread(&childRes, sizeof(int), 1, fp);
    printf("\nParent: child result: %d\n", childRes);

    if(++n>5){
      printf("Parent: work done, bye bye\n");
      unlink("/tmp/ipoc.txt");
      kill(0, SIGTERM);
    }
    printf("Parent: waiting for child\n\n");
    kill(pid, SIGUSR1);
} }
```

Example 1... (child)

```
void child(FILE *fp){
  int value;

  while(1){
    sleep(1);
    value = random()%100;
    rewind(fp);
    fwrite(&value, sizeof(int), 1, fp);
    printf("Child: waiting for parent\n\n");
    signal(SIGUSR1, action);
    kill(getppid(), SIGUSR1);
    pause();
  }
}
```

Part II
Unnamed Pipes

Unnamed pipes

Unnamed pipes

- (Unnamed) Pipes (vs FIFO), known as `pipe`, are a mechanism for Inter-Process Communication (IPC).
- Pipes are used by shells to connect one utility's standard output with the standard input of another utility.
- Example: `ps -ef | grep bash | wc -l`
- A process creates a pipe, forks then, uses the pipe to exchange information with its child.

Limitations

Pipes are the oldest form of Unix IPC. They have two limitations:

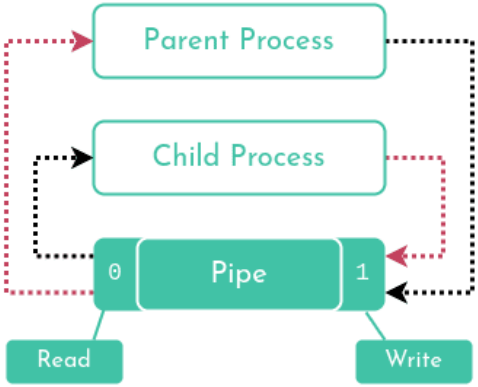
- They are half-duplex: data flows in one direction only.
- They can be used only between processes that have a common ancestor.

Pipes

A pipe is a connection between two processes, such that the standard output from one process becomes the standard input of the other process. In UNIX Operating System, Pipes are useful for communication between related processes(Inter-Process Communication – IPC):

- Pipe is one-way communication only. i.e., we can use a pipe such that one process writes to the pipe, and the other process reads from the pipe. It opens a pipe, which is an area of main memory that is treated as a ***"virtual file"***.
- The pipe can be used by the creating process, as well as all its child processes, for reading and writing. One process can write to this "virtual file" or pipe and another related process can read from it.
- If a process tries to read before something is written to the pipe, the process is suspended until something is written.
- The pipe system call finds the first two available positions in the process's open file table and allocates them for the read and write ends of the pipe

Pipes



such that the standard output
of the other process. In UNIX
nication between related

can use a pipe such that One
ess reads from the pipe. It opens a
s treated as a ***"virtual file"***.

, as well as all its child processes,
e to this "virtual file" or pipe and

written to the pipe, the process is

ble positions in the process's open
write ends of the pipe

The pipe() system call

pipe()

Synopsis: `int pipe(int fd[2])`
returns 0 when successful and -1 otherwise
`pipe()` creates a pipe and returns two file descriptors:
`fd[0]` and `fd[1]`
`fd[0]` is open for reading,
`fd[1]` is open for writing.

A **pipe** is a one-way communication channel between two related processes.

Rules when reading from and writing to a pipe

Rules

When reading from or writing to a pipe, the following rules apply:

1. If a process reads from a pipe whose write-end has been closed, after all data has been read, `read()` returns 0.
2. If a process reads from an empty pipe whose write-end is still open, it sleeps until some input becomes available.
3. If a process tries to read from a pipe more bytes than are present, `read()` reads all available bytes and returns the number of bytes read.
4. If a process writes to a pipe whose read-end has been closed, the write operation fails and the writer process receives a `SIGPIPE`.

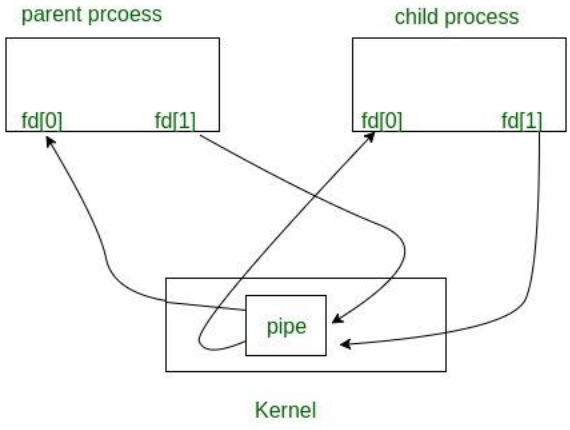
In case of multiple processes writing to the same pipe, a `write` of up to `PIPE_BUF` bytes is guaranteed to be atomic.
→ data from different writer processes will not be interleaved.

`fork()` and Pipes

`fork()` in any process:

- The file descriptors remain open across the child process.
- The file descriptors remain open across parent process.
- If we call `fork()` after creating a pipe, then the parent and child can communicate via the pipe.

`fork()` and Pipes



oss the child process.

oss parent process.

then the parent and child

University of Windsor

Example 2: Child is a writer, Parent is a reader...(main) (pipe_ex1.c)

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>


void child(int *);
void parent(int *);

int main(int argc, char *argv[]){
  int fd[2];

  if(pipe(fd) == -1)
    exit(1);

  if(fork() == 0)
    child(fd);
  else
    parent(fd);
  exit(0);
}
```

Example 2: Child is a writer, Parent is a reader... (parent)

```
void parent(int *fd){
  char ch;

  close(fd[1]);
  printf("Child has sent the message:\n");
  do{
    read(fd[0], &ch, 1);
    printf("%c", ch);
    if(ch == '\n')
      break;
  }while(1);
}
```

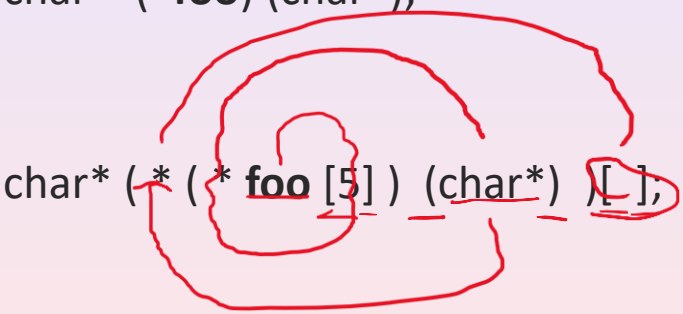
Example 2: Child is a writer, Parent is a reader... (child)

```
void child(int *fd){
  char message[255]="Hello, here is my data...\n";

  close(fd[0]);

  write(fd[1], message, 26);
}
```

char* (*foo) (char*);



Pipe1.c ←Homework
Pipe2.c
Pipe3.c
Pipesize.c

Example 3: Pipe1.c

```
#include <unistd.h>
#include <stdio.h>
#include <stdlib.h>

#define MSGSIZE 16

char *msg1 = "hello, world #1";
char *msg2 = "hello, world #2";
char *msg3 = "hello, world #3";

//a pipe for one process only
int main() {
    char inbuf[MSGSIZE];
    int p[2], j;

    if (pipe(p) == -1) {
       perror("pipe call");
       exit(1);
    }

    write(p[1], msg1, MSGSIZE);
    write(p[1], msg2, MSGSIZE);
    write(p[1], msg3, MSGSIZE);

    for (j = 0; j <3; j++) {  //change 3 to 5
         read(p[0], inbuf, MSGSIZE);
         printf("%s\n", inbuf);
    }

    exit(0);
}
```

The dup2() system call

dup2()

Synopsis: `int dup2(int fd, int fd2);`

The `dup2()` function causes the file descriptor `fd2` to refer to the same file as `fd`.

- The `fd` argument is a file descriptor referring to an open file.
- `fd2` is a non-negative integer less than the current value for the maximum number of open file descriptors allowed the calling process.
- If `fd2` already refers to an open file, not *fd*, it is closed first.
- If `fd2` refers to `fd`, or if `fd` is not a valid open file descriptor, `fd2` will not be closed first.
- If successful, `dup2()` returns a non-negative integer representing the file descriptor `fd2`.
- Otherwise, -1 is returned.

Just curious, what is `dup()` ?

After the call: **`dup2(fd, 0);`**
reading from `stdin` will mean reading from the file whose descriptor is `fd`.

Example 3: Implementing the shell pipe mechanism (join.c)

See the **join.c** code posted. ←Homework

Example 4: 2-player game with a referee...(main) (game.c)

```
int main(int argc, char *argv[]){  int
  fd1[2], fd2[2], fd3[2], fd4[2];  char
  turn='T';

  printf("This is a 2-player game with a referee\n");
  pipe(fd1);
  pipe(fd2);
  if(!fork())
    player("TOTO", fd1, fd2);

  close(fd1[0]); // parent only write to pipe 1
  close(fd2[1]);   // parent only reads from pipe 2

  pipe(fd3);
  pipe(fd4);
  if(!fork())
    player("TITI", fd3, fd4);

  close(fd3[0]); // parent only write to pipe 3
  close(fd4[1]);   // parent only reads from pipe 4

  while(1){
    printf("\nReferee: TOTO plays\n\n");
    write(fd1[1], &turn, 1);
    read(fd2[0],  &turn, 1);

    printf("\nReferee: TITI plays\n\n");
    write(fd3[1], &turn, 1);
    read(fd4[0],  &turn, 1);
  }
 }
```

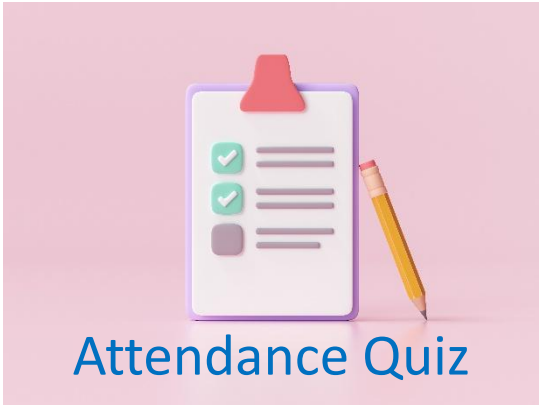
Example 4: 2-player game with a referee...(player)

```
void player(char *s, int *fd1, int *fd2){
  int points=0;
  int dice;
  long int ss=0;
  char turn;
  close(fd1[1]);
  close(fd2[0]);
  while(1){
    read(fd1[0], &turn, 1);
    printf("%s: playing my dice\n", s);
    dice =(int) time(&ss)%10 + 1;
    printf("%s: got %d points\n", s, dice);
    points+=dice;
    printf("%s: Total so far %d\n\n", s, points);
    if(points >= 50){
      printf("%s: game over I won\n", s);
      kill(0, SIGTERM);
    }
    sleep(5);     // to slow down the execution
    write(fd2[1], &turn, 1);
  }

}
```

Part V
Attendance Quiz (Not graded)

Quiz



Thank you!
Questions?

Dr. Andreas S. Maniatis
Assistant Professor
Andreas.Maniatis@UWindsor.ca
http://www.linkedin.com/in/andreasmaniatis